Supplementary Methodology and Complete Clean Results: Pre- and Post-Encoding Augmentation for Spiking Neural Networks

Manu Singh Bist

Data Science Group, University of Hildesheim
Hildesheim, Germany

Abstract. This companion document describes the implementation of the data augmentation methods used in the main ESANN paper. It explains where each method acts in a spiking network, how its parameters and random operations are defined, and how labels and spike values are handled. It also records the training and evaluation protocol. Small examples are included to make the transformations easier to follow. Complete clean top-1 classification results from the accompanying thesis are also provided for the methods retained in the main paper. This optional companion is separate from the reviewed ESANN submission.

1 Scope and notation

Main paper: Open the six-page ESANN manuscript.

The main paper compares augmentation before spike encoding with augmentation on features produced by an early spiking stage. This supplement gives the full transformation rules so that the methods can be understood without relying on the short descriptions in the main manuscript. The description follows the accompanying master's thesis. It does not introduce a different model or a new experiment.

1.1 Input, spike representation, and network stages

For a static image, let $\mathbf{x} \in \mathbb{R}^{C\times H\times W}$ be the input. For events, the data loader produces binned event frames, commonly arranged as $\mathbb{R}^{B\times T\times C\times H\times W}$. A spike encoder E produces a time-dependent representation. We write the ideal binary output as

$$\mathbf{X} = E(\mathbf{x}) \in \{0,1\}^{T\times B\times C'\times H'\times W'}, \tag{1}$$

where B denotes batch size, T is the number of simulation steps, C' is the number of encoded channels, and $H' \times W'$ is the encoded spatial grid. We generally put time first for internal spike tensors. Some implementations and data loaders put batch first. We state the expected layout whenever it matters.

The first leaky integrate-and-fire (LIF) stage integrates input over time and emits thresholded spikes. A representative discrete form is

$$u_t = \lambda u_{t-1} + W z_t - \vartheta s_{t-1}, \qquad s_t = \mathbb{I}[u_t \geq \vartheta]. \tag{2}$$

Here λ controls leak, ϑ is the threshold, and s_t is the output spike. The SNN is trained with surrogate gradients, since the threshold function does not have

a useful ordinary derivative [1]. Equation (2) is a compact explanation of the neuron operation, not a claim that every implementation uses the same reset update.

1.2 Where augmentation is inserted

Pre-encoding augmentation changes the image or event frames before the encoder:

$$\widetilde{\mathbf{x}} = A_{\text{pre}}(\mathbf{x}), \qquad \widehat{y} = F(E(\widetilde{\mathbf{x}})). \tag{3}$$

The encoder therefore learns from changed pixel or event values. The spatial patch methods, IPMix, and LayerMix are included in this family.

Post-encoding augmentation changes a feature tensor produced by an early encoder or spiking layer. If F_1 denotes the early stage and F_2 the remaining network, then

$$\mathbf{X}_1 = F_1(E(\mathbf{x})), \quad \widetilde{\mathbf{X}}_1 = A_{\text{post}}(\mathbf{X}_1), \quad \widehat{y} = F_2(\widetilde{\mathbf{X}}_1). \tag{4}$$

The first stage sees an unmodified input, while later layers learn from modified features. In the code, these insertions are implemented with forward hooks. The exact hook differs by method: the patch encoder, its embedding output, or an early convolutional layer may be used. Thus, "after the first LIF" describes the representative position, not one universal module for all backbones.

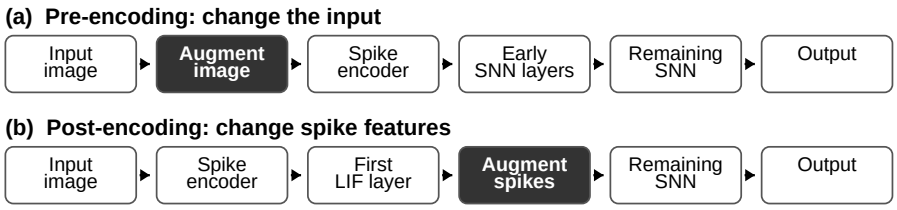


Fig. 1: Compact view of the two locations. Pre-encoding methods change input values; post-encoding methods change intermediate spiking features. The transformations are used mainly during training.

1.3 Stochastic gates and evaluation

Each augmentation has its own probability p. During training, it is applied when a Bernoulli gate with success probability p is active. Some gates are per batch, some per sample, and some per feature map. We describe that choice for each method below. Unless explicitly enabled otherwise, all augmentations are switched off when the network is evaluated. TimeMix has an optional evaluation-time flag, but the augmentation-only setting leaves it disabled.

A key distinction is that not every post-encoding feature is binary. Permutation and binary masking preserve $\{0, 1\}$ values, but linear blending can create

real-valued activations. Subsequent spiking layers turn them back into spikes. The implementation therefore should not assume that every augmented tensor is binary.

2 Patch-based augmentation before encoding

These methods work on static images or binned event tensors. They keep the input's overall spatial resolution unchanged. For events, their handling of the time axis differs, as explained below.

2.1 PatchShuffle

PatchShuffle changes the ordering of pixel values within small, non-overlapping patches [2]. Let a patch have spatial size $H_p \times W_p$. For each patch position (i, j), draw a permutation $\pi_{i,j}$ of its $H_p W_p$ flattened positions and set

$$\widetilde{\mathbf{x}}_{b,c,i,j} = \operatorname{unvec}\left(\Pi_{b,i,j}\,\operatorname{vec}(\mathbf{x}_{b,c,i,j})\right), \tag{5}$$

where $\Pi_{b,i,j}$ is the corresponding permutation matrix. For the input-stage version, the same permutation is used for all channels of a sample. For event frames, the batch and time dimensions are flattened, so each frame can be shuffled independently. The Bernoulli gate is drawn per sample or frame.

The thesis uses 2×2 patches and a probability of 0.05. If a patch does not fit at the right or bottom border, only complete patches are shuffled; the leftover border stays unchanged. A patch permutation preserves the set of values within that patch. For a binary tensor, it also preserves the patch spike count. An optional dual-batch mode concatenates a clean copy with an augmented copy, but that mode is not required to define the core transform.

2.2 PatchDropout

Pre-encoding PatchDropout removes complete image regions by setting them to zero [3]. Given square patch width P, define the patch grid

$$G_h = \lfloor H/P \rfloor, \quad G_w = \lfloor W/P \rfloor, \quad N = G_h G_w. \tag{6}$$

With keep rate r, keep $m = \max(1, \operatorname{round}(rN))$ randomly chosen patches. A mask M is one on kept regions and zero on removed regions:

$$\widetilde{\mathbf{x}}_{b,c,h,w} = M_{b,h,w}\,\mathbf{x}_{b,c,h,w}. \tag{7}$$

The same spatial mask is shared over the channels. For event input, it is also shared over the time bins of one sample, so a removed region stays removed throughout the sample. The thesis default is patch size $P = 16$, keep rate $r = 0.95$, and an always-active application gate. For a 64×64 input, this gives sixteen possible patches and keeps fifteen. This adaptation differs from the original transformer method because tokens are not physically deleted; the tensor size remains fixed.

2.3 PatchMix

PatchMix blends patches from different locations *within the same sample* [4]. Split the image into a $G \times G$ grid, label the patches $P_1, \ldots, P_{G^2}$, and sample a permutation π of these patch indices. Draw a switch $r_i \sim \text{Bernoulli}(p)$ for each patch. If $r_i = 0$, set $\lambda_i = 1$. Otherwise draw $\lambda_i \sim \text{Beta}(a, b)$ and write

$$\widetilde{P}_i = \lambda_i P_i + (1 - \lambda_i) P_{\pi(i)}. \tag{8}$$

The thesis uses $G = 2$, $p = 0.5$, and $a = b = 0.2$. For event tensors, the same partner permutation and mixing weights are shared across channels and time bins of one sample. As both patches belong to the same input, the target class is unchanged. Note that the blended values need not be binary, even if the source feature maps are binary.

3 Temporal augmentation after encoding

These methods operate on $\mathbf{X} \in \mathbb{R}^{T \times B \times C \times H \times W}$. None of them changes the spatial layout. TimeShuffle, Local TimeShuffle, TimeMask, and Temporal Jitter keep the tensor binary when the input is binary. TimeMix can produce continuous values.

3.1 TimeShuffle

TimeShuffle draws a random permutation π_b of the time indices for each sample and reorders its frames:

$$\widetilde{\mathbf{X}}_{t,b,c,h,w} = \mathbf{X}_{\pi_b(t),b,c,h,w}. \tag{9}$$

The default uses a separate permutation for each sample, shared over its channels. The implementation also allows one permutation for the entire batch, or separate permutations for each sample-channel pair. The last option changes the cross-channel timing more strongly. Since this operation only reorders entries, it does not change the total spike count of the tensor. Its purpose is to reduce dependence on one fixed temporal ordering.

3.2 TimeMask

TimeMask removes selected time steps, rather than individual pixels. Let r be the masked fraction and $k = \max(1, \lceil rT \rceil)$. Sample a mask $m \in \{0, 1\}^T$ with exactly k zeros. Then

$$\widetilde{\mathbf{X}}_{t,b,c,h,w} = m_{t,b,c} \mathbf{X}_{t,b,c,h,w}. \tag{10}$$

The mask's batch and channel indices are omitted when it is shared. The default is a per-sample mask shared across channels, with $r = 0.25$; alternatives are one mask for the whole batch or a separate mask for every sample-channel pair. When $T = 4$ and $r = 0.25$, one complete frame is removed. Unlike TimeShuffle, this operation lowers the spike count at the augmented layer.

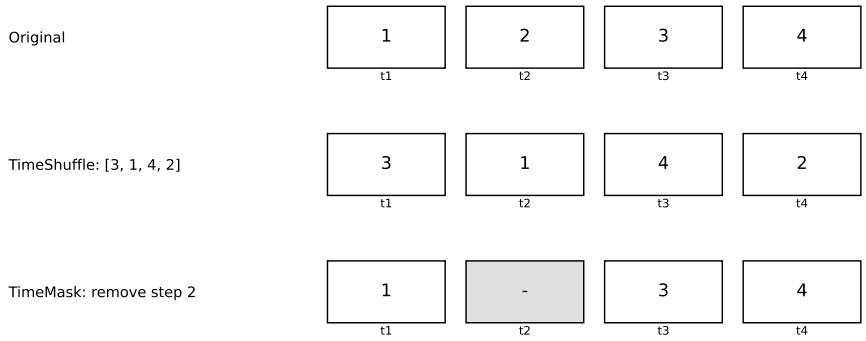


Fig. 2: A small sequence example. Each numbered cell represents one complete feature frame. TimeShuffle changes their order without removing frames; TimeMask sets the selected frame to zero. The diagram is illustrative, not an experimental observation.

3.3 TimeMix

TimeMix draws on the Temporal Shift Module idea [5]. Divide channels into G groups, with group width $C_{\text{fold}} = C/G$. Choose split indices $g_1 < g_2$. Channels in the first range are shifted left in time, channels in the second range are shifted right, and the remaining channels are unchanged. The shifted tensor Z is

$$Z_{t,c} = \begin{cases} X_{t+1,c}, & c < g_1 C_{\text{fold}}, \\ X_{t-1,c}, & g_1 C_{\text{fold}} \le c < g_2 C_{\text{fold}}, \\ X_{t,c}, & \text{otherwise}, \end{cases} \tag{11}$$

with all spatial and batch indices understood. Out-of-range time positions are filled with zero rather than wrapped around. The output combines shifted and original features:

$$\widetilde{\mathbf{X}} = \mathbf{X} + \alpha Z. \tag{12}$$

The default scaling is $\alpha = 0.5$ and the default group count is $G = 32$ where compatible with the channel dimension. Random group splits are the default, and fixed splits are also available. The result can contain values such as 0, 0.5, 1, or 1.5, so this particular method is not binary-preserving. In the thesis it is used as a training-time augmentation, even though a fixed time shift can also be used as a permanent architecture component.

3.4 Local TimeShuffle

Full TimeShuffle can remove useful ordering in event sequences. Local TimeShuffle breaks the time axis into windows of length L and shuffles frames only inside

each window. Let π_L be the concatenation of independent random permutations for all windows. Then

$$\widetilde{\mathbf{X}}_{t,b,c,h,w} = \mathbf{X}_{\pi_L(t),b,c,h,w}. \tag{13}$$

The last window may have fewer than L elements. In the described time-major implementation, one windowed permutation is shared across all samples and channels. For example, $[1, 2, 3, 4 \mid 5, 6, 7, 8]$ could become $[3, 1, 4, 2 \mid 6, 8, 5, 7]$. No index moves between windows. If $L = 1$, nothing changes; if $L = T$, this becomes a full shuffle. The total spike count is unchanged.

3.5 Temporal Jitter

Temporal Jitter uses a circular shift rather than an arbitrary permutation. Choose k uniformly from $\{-K, \ldots, K\}$ and set

$$\widetilde{\mathbf{X}}_{t,b,c,h,w} = \mathbf{X}_{(t-k) \bmod T,b,c,h,w}. \tag{14}$$

In the global variant, the whole batch shares k. In the per-sample variant, each example has its own k_b. Because the shift wraps around, all the original spikes are retained. This differs from TimeMix, which zero-pads time boundaries and combines shifted and unshifted tensors.

4 Patch-based augmentation after encoding

The next three methods act on feature maps after an encoder or an early feature block. Their locations depend on what the model exposes. The first three use the same basic rule as the pre-encoding methods but operate on learned features rather than input pixels.

4.1 Post-encoding PatchShuffle

Equation (5) also applies to post-encoder features. The thesis uses 2×2 patches and a gate probability of 0.05. The main difference is the scope: a separate Bernoulli decision and patch permutations may be drawn for every sample-channel feature map, rather than one decision per raw image. For the convolutional implementation, a training-only forward hook is attached to the convolutional feature maps. Permutation preserves the values and the local spike count when the features are binary.

4.2 Post-encoding PatchDropout

For a feature frame with $N = HW$ spatial locations, sample a set of

$$m = \max(1, \operatorname{round}(rN)) \tag{15}$$

locations to keep. A zero-one spatial mask M gives $\widetilde{\mathbf{X}}_{t,b,c,h,w} = M_{t,b,h,w}\mathbf{X}_{t,b,c,h,w}$. Every channel in a frame sees the same mask. Here the B and T axes are flattened, so each frame can have its own spatial mask. Besides a fixed keep rate

r, the implementation supports a randomly chosen rate $r \sim \mathcal{U}(r_{\min}, 1)$ on each forward pass.

The difference from the original PatchDropout for vision transformers is important: the selected locations are zeroed *in place*. They are not gathered into a shorter token sequence. The spiking backbone still receives its expected fixed-size feature grid, so this form does not directly reduce the size of the attention operation. It changes the signal and the later spike activity instead.

4.3 Post-encoding PatchMix

The spatial grid is divided into G^2 patches, and Equation (8) is applied to the learned feature map of each sample. The same sampled permutation and per-patch mixing weight are used across time steps and channels of that sample. As in the pre-encoding version, $G = 2$, $p = 0.5$, and $\mathrm{Beta}(0.2, 0.2)$ are the default settings in the thesis. The hook is attached to the patch embedding and disabled during evaluation. Labels stay unchanged because patch partners come from the same sample.

5 Spike-map repair

5.1 Spatial Hole Filling

Spatial Hole Filling (SHF) repairs an isolated zero pixel when all eight surrounding pixels are active. Let K_{2D} be a 3×3 kernel of ones except for a zero at the centre:

$$K_{2D} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}. \tag{16}$$

For each frame and channel, count the active neighbours with a zero-padded convolution, $N_8 = K_{2D} * X$. The output rule is

$$\widetilde{\mathbf{X}}_{t,c,h,w} = \begin{cases} 1, & X_{t,c,h,w} = 0 \text{ and } (N_8)_{t,c,h,w} = 8, \\ X_{t,c,h,w}, & \text{otherwise.} \end{cases} \tag{17}$$

The method treats any nonzero input as active for its test. The strict condition avoids adding spikes to ordinary open boundaries. Zero padding also means border pixels do not qualify. The operation has a per-sample training-time probability gate, with default $p = 1$.

5.2 Spatiotemporal Hole Filling

Spatiotemporal Hole Filling (STHF) checks a $3 \times 3 \times 3$ neighbourhood over time and space. Its convolution kernel contains 26 ones and a zero at the centre. If $N_{26} = K_{3D} * X$, the update is

$$\widetilde{\mathbf{X}}_{t,c,h,w} = \begin{cases} 1, & X_{t,c,h,w} = 0 \text{ and } (N_{26})_{t,c,h,w} = 26, \\ X_{t,c,h,w}, & \text{otherwise.} \end{cases} \tag{18}$$

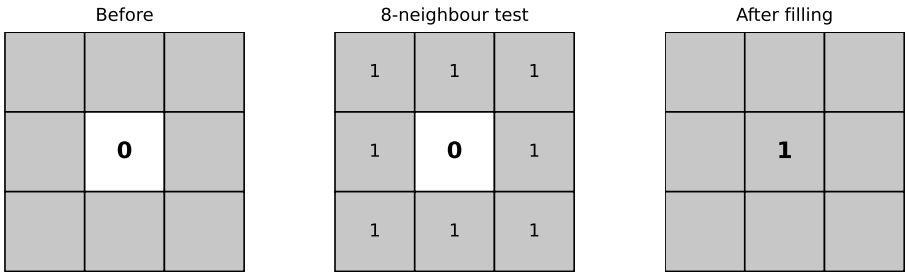


Fig. 3: Illustration of Spatial Hole Filling. The centre is zero, but all eight neighbours are active, so the centre is changed to one. A partially surrounded zero would stay unchanged.

The kernel operates independently for each channel. Zero padding is applied along time, height, and width, so the first and last time slices and other volume boundaries do not get filled. SHF only checks the current spatial frame; STHF requires agreement across three adjacent time steps and can therefore repair a narrower kind of gap. The default training-time gate is also $p = 1$. These are deterministic repair rules once their Bernoulli gate is active, rather than learned denoisers.

6 Binary-safe and permutation-based mixing

6.1 Binary Spike CutMix (BS-CutMix)

A simple average of two spike maps produces fractional values. BS-CutMix instead draws a binary mask and chooses one of the two source spikes at each position. Let $X \in \{0,1\}^{T\times B\times C\times H\times W}$ and let X' be its partner. A mask $M \in \{0,1\}^{T\times B\times 1\times H\times W}$ with $\Pr(M = 1) = 0.5$ is broadcast over channels. The result is

$$\widetilde{\mathbf{X}} = M \odot X' + (1 - M) \odot X. \tag{19}$$

This is an element-wise choice, not a rectangular CutMix region. As M, X, and X' are binary, $\widetilde{\mathbf{X}}$ is also binary.

The partner selection keeps the target label valid. If *all* samples in a batch have the same label, X' is built by a random batch permutation. If the permutation is the identity, it is shifted by one position to avoid a no-op. If the batch contains more than one class, X' is instead built by a random permutation of the time axis *inside each sample*. Thus no information is taken from a different class in this usual multi-class case. The original hard label is kept in both cases. The thesis gives application probability $p = 0.5$ and disables the method at evaluation. The method is not run together with ordinary MixUp or CutMix, whose soft-label handling would be inconsistent with its hard-label design.

6.2 Per-Axis Permutation Mix (PAPMix)

PAPMix builds a scrambled partner by independently permuting the axes of an input tensor. The routine accepts both static-image batches $x \in \mathbb{R}^{B\times C\times H\times W}$ and event-frame batches $x \in \mathbb{R}^{B\times T\times C\times H\times W}$. Although it is grouped with post-encoding or time-aware operations in the thesis, its attachment point is the model input in the 4D static-image path; in the 5D event path, a time axis is already present in the binned representation. This distinction matters when comparing placements.

For the 5D case, independently draw permutations $\pi_B, \pi_T, \pi_C, \pi_H, \pi_W$ and set

$$x^{\pi}_{b,t,c,h,w} = x_{\pi_B(b),\pi_T(t),\pi_C(c),\pi_H(h),\pi_W(w)}. \tag{20}$$

The time permutation is absent for a 4D input. The output is a fixed-weight blend,

$$x^{\text{out}} = \alpha x + (1-\alpha)x^{\pi}. \tag{21}$$

Unlike PatchMix and BS-CutMix, this operator can mix different samples across classes. It therefore also mixes the targets. With K classes and smoothing coefficient s, turn an integer label into a distribution q_b whose correct class receives $1 - s + s/K$ and every other class receives s/K. Then

$$q^{\text{out}} = \alpha q + (1-\alpha)q[\pi_B]. \tag{22}$$

If the input labels are already soft distributions, the implementation keeps them rather than converting the integer labels again. The thesis uses $\alpha = 0.5$ and considers application probabilities $p \in \{0, 0.25, 0.5, 1\}$. Both the input transformation and the label transformation must be used together.

6.3 Previously published reference methods

Three reference methods help put the placement experiments in context. Random Spike Masking (RSM) drops spike activity during training [6]. IPMix combines image-, patch-, and pixel-level transformations, while LayerMix uses fractal-based image synthesis and mixing [7, 8]. These are treated as comparison methods rather than new contributions. Their full original algorithms are defined in the cited papers; no new formula is claimed here for their internals.

7 Augmentation pipeline and implementation choices

7.1 Order of transformations

A combined augmentation block can run temporal operations first, followed by spatial feature-map operations. The thesis provides the following sequence as a framework for six methods. Each step is optional and governed by its own gate. A typical single-method experiment enables only the method under study.

Stage	Operation
1	Optionally apply TimeShuffle to the temporal axis.
2	Optionally apply TimeMask to selected time steps.
3	Optionally apply TimeMix, using channel shifts followed by residual addition.
4	Optionally apply PatchShuffle within local feature patches.
5	Optionally apply PatchDropout while keeping the spatial tensor shape.
6	Optionally apply PatchMix between patches from the same sample.

The temporal stage comes first because it acts only along T and does not change where spatial patches are located. The spatial stage then sees the temporally modified representation. Mixing is applied after simple permutation or masking within its stage. The ordering is an implementation choice, not a mathematical assertion that all operations commute.

7.2 Shape, spike values, and labels

Most spatial and temporal methods keep T, B, C, H, and W unchanged. A practical implementation must also keep batch-major and time-major conventions consistent at hooks. Simple permutations are value preserving; masks remove spikes; hole-filling adds spikes; and linear transforms such as TimeMix, PatchMix, and PAPMix may create real-valued features. Only methods that mix samples across class labels require soft targets, which is the case for PAPMix. PatchMix stays inside one sample, and BS-CutMix chooses label-safe partners, so both retain hard labels.

8 Experimental protocol and evaluation measures

8.1 Datasets and architectures

Static-image classification uses CIFAR-10, CIFAR-100, and Tiny-ImageNet. Event-camera classification uses CIFAR-10-DVS and DVS128 Gesture [9, 10]. The two architectures are the Spike-driven Transformer (SDT) [11] and membrane-shortcut ResNet (MS-ResNet) [12]. The thesis contains a wider SDT evaluation; not all MS-ResNet experiments are available for the other datasets.

8.2 Training settings

Static images are normalized with dataset channel statistics. Event streams are converted to binned frames and use event-appropriate augmentation. The geometric baseline uses random crop and horizontal flip. A separate "no augmentation" setting removes this baseline augmentation. Other than the selected augmentation, the core optimization settings are fixed within each backbone. Table 1 records the settings in the thesis.

All settings are trained from scratch. The thesis reports results averaged over five seeds for CIFAR-100 and three seeds for the other datasets unless a run is marked as single-run. For transparency, the thesis also states that it selected checkpoints using the best clean *test* accuracy. Selecting with the test set can

Table 1: Common training settings reported in the thesis.

Setting	SDT	MS-ResNet
Optimizer	SGD	SGD
Momentum	0.9	0.9
Initial learning rate	0.1	0.1
Schedule	Cosine	Cosine
Weight decay	5×10^{-4}	5×10^{-4}
Batch size	128	128
Training epochs	310	100
Simulation steps T	4	4

bias a performance estimate. A stronger final submission protocol would choose checkpoints using a separate validation set and evaluate the test set once. The protocol in the paper should be updated only if the experiments themselves are changed; this supplement does not imply that retraining has already taken place.

8.3 Clean-accuracy evaluation

Clean top-1 accuracy is the fraction of correctly classified examples on the unchanged test set:

$$\mathrm{Acc}_{\mathrm{clean}} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}[\widehat{y}_i = y_i]. \tag{23}$$

The clean scores below are reported on the original classification test sets, without input corruptions.

9 Complete clean classification results

Table 2 reports the clean top-1 test accuracy for all methods retained in the main paper, using results from the accompanying thesis. SDT-100 and MS-100 denote the two backbones on CIFAR-100. SDT-10 and Tiny denote SDT on CIFAR-10 and Tiny-ImageNet. DVS-10 and Gesture denote SDT on CIFAR-10-DVS and DVS128 Gesture. Only the CIFAR-100 benchmark has MS-ResNet results in this study. The temporal methods use the insertion points of their corresponding experiments, which vary between backbones, rather than one shared stage.

Table 2: Complete clean top-1 accuracy (%) for the retained augmentations. Bold indicates the best result in each dataset.

Method	SDT-100	MS-100	SDT-10	Tiny	DVS-10	Gesture
Baselines						
No augmentation	68.39	57.10	90.07	40.46	63.23	94.09
Standard baseline	78.40	62.05	95.23	61.21	70.29	**96.87**
Spatial and masking						
RSM	76.13	59.47	93.75	51.78	67.12	95.13
PatchDropout	78.67	63.90	95.74	53.98	68.70	94.09
PatchShuffle	78.78	62.20	95.60	56.88	69.25	**96.87**
Spatial Hole Fill.	79.06	62.03	95.63	60.86	69.56	96.18
PatchMix	79.29	54.74	**95.89**	60.32	68.22	95.13
Temporal						
TimeMix	78.69	61.94	95.69	61.45	68.67	96.18
Temporal Jitter	78.90	62.55	95.46	57.32	68.46	94.96
Local TimeShuffle	79.15	60.70	95.81	56.30	**72.22**	62.50
TimeShuffle	79.45	63.42	95.87	**61.92**	70.78	**96.87**
TimeMask	**79.56**	62.07	95.72	61.25	72.00	**96.87**
Spatiotemporal						
PAPMix	78.80	61.30	95.47	59.10	67.11	95.14
STHF	79.17	**64.92**	95.80	61.33	70.00	94.44
BS-CutMix	79.38	59.53	95.81	57.24	69.44	96.53
Published references						
LayerMix	72.64	59.94	92.63	55.23	70.22	96.22
IPMix	74.43	63.72	94.07	56.17	71.56	89.23
Post-encoding patch variants						
PatchDropout	78.93	59.83	95.41	53.89	68.11	93.75
PatchShuffle	78.44	53.65	95.79	58.64	68.78	96.42
PatchMix	78.99	61.51	95.74	60.82	70.22	96.52

Backbone differences. On CIFAR-100, SDT reaches 79.56% with TimeMask against a baseline of 78.40%, while MS-ResNet reaches 64.92% with Spatiotemporal Hole Filling against 62.05%. The better method therefore depends on the network architecture. On CIFAR-10, SDT reaches 95.89% with pre-encoding PatchMix compared with the 95.23% baseline. On Tiny-ImageNet, TimeShuffle reaches 61.92%, compared with the baseline of 61.21%.

Placement and event data. PatchMix on SDT CIFAR-100 reaches 79.29% before encoding and 78.99% after encoding; on MS-ResNet, the corresponding values are 54.74% and 61.51%. On CIFAR-10-DVS, Local TimeShuffle reaches

72.22% compared with the 70.29% baseline, whereas on DVS128 Gesture it drops to 62.50% and the baseline is 96.87%. This shows that temporal changes should reflect the event sequence structure.

Interpretation limits. The results are clean classification scores only, not corruption or energy comparisons. The source thesis reports checkpoint selection using clean test accuracy, which can bias estimates. Validation-based checkpoint selection is recommended for future experiments.

10 Reproducibility notes

For a new implementation, the following details should be checked against the training configuration used to produce each result: exact augmentation hook; whether the random gate is per batch, sample, frame, or feature map; treatment of time-major versus batch-major tensors; whether the output is still binary; and whether the labels need mixing. When applying methods sequentially, record both the order and the active probabilities. Results should be reported with random seeds, clean classification accuracy, and the checkpoint selection rule. The paper's main text remains self-contained. This companion provides the full clean result tables and the implementation details, but no new experiments or extra claims essential to the paper.

References

[1] E. O. Neftci, H. Mostafa, and F. Zenke. Surrogate gradient learning in spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019.

[2] G. Kang et al. PatchShuffle regularization. arXiv:1707.07103, 2017.

[3] Y. Liu et al. PatchDropout: Economizing vision transformers using patch dropout. *WACV*, 2023.

[4] Y. Hong and Y. Chen. PatchMix: Patch-level mixup for data augmentation in convolutional neural networks. *Knowledge and Information Systems*, 66:3855–3881, 2024.

[5] J. Lin, C. Gan, and S. Han. TSM: Temporal shift module for efficient video understanding. *ICCV*, 2019.

[6] Z. Wang et al. Masked spiking transformer. *ICCV*, 2023.

[7] Z. Huang et al. IPMix: Label-preserving data augmentation method for training robust classifiers. *NeurIPS*, 2023.

[8] H. M. Ahmad et al. LayerMix: Enhanced data augmentation through fractal integration for robust deep learning. arXiv:2501.04861, 2025.

[9] H. Li et al. CIFAR10-DVS: An event-stream dataset for object classification. *Frontiers in Neuroscience*, 11:309, 2017.

[10] A. Amir et al. A low power, fully event-based gesture recognition system. *CVPR*, 2017.

[11] M. Yao et al. Spike-driven transformer. *NeurIPS*, 2023.

[12] Y. Hu, H. Tang, and G. Pan. Advancing spiking neural networks toward deep residual learning. *IEEE TNNLS*, 2024.